

Vereins-OS / M75-Manager — Statusbericht & Ausbau-Empfehlungen

Erstellt: 29. Juni 2026 · **Autor:** Cascade (KI-Assistenz) · **Basis:** Code-Analyse des Repos `Netjogger58/Vereins-OS` **Zweck:** Ehrlicher Ist-Zustand (fertig / ausbaufähig / nur Idee), Bewertung von Struktur, Komplexität, Aufbau & Visuellem — plus konkrete Anregungen inkl. KI-/Automatisierungs-/Monitoring-Konzept.

0. Kurzfazit (TL;DR)

Vereins-OS ist **kein Prototyp mehr, sondern eine ernsthafte, große Anwendung**: ~16.700 Zeilen App-Code, **233 API-Endpunkte**, **80 Datenbank-Tabellen**, 35 Seiten, 47 UI-Komponenten. Der **handballsportliche Kern** (Spiele, Statistik, FLH-Import, Teams, Mitglieder, Finanzen, Training, Check-in) ist real implementiert und nutzbar.

Die wichtigste Erkenntnis: **Das Datenmodell ist der Oberfläche weit voraus**. Für viele geplante Module (Schiedsrichter, Verletzungen/Reha, Umfragen, Fahrgemeinschaften, Gegner-Scouting, Saison-Archiv, Live-Ticker) **existieren bereits Tabellen, aber noch keine bzw. nur Platzhalter-UI**. Das ist eine sehr gute Ausgangslage: der teure Teil (Schema-Design) ist gemacht.

Größtes Risiko aktuell: **Reife-Gefälle** zwischen voll ausgebauten und Platzhalter-Seiten (9 identische 89-Zeilen-Stubs), plus operative Themen (SQLite/WAL, Demo-Passwörter, fehlende KI-/Automations-Schicht, die im Plan steht, aber noch nicht läuft).

1. Kennzahlen (gemessen am Code)

Metrik	Wert	Anmerkung
API-Endpunkte (<code>routes.ts</code>)	233	<code>app.get/post/put/patch/delete</code>
Datenbank-Tabellen (<code>schema.ts</code>)	80	<code>Drizzle sqliteTable</code>
Frontend-Seiten	35	<code>client/src/pages/*.tsx</code>
<code>useQuery</code> -Aufrufe	93	React-Query-Anbindung
<code>shadcn/ui</code> -Komponenten	47	<code>components/ui</code>
Backend-Code	~7.990 Zeilen	<code>server/*.ts</code>
Frontend-Seiten-Code	~8.713 Zeilen	nur <code>pages/</code> , ohne UI-Lib
<code>storage.ts</code>	~160 KB	zentrale Datenzugriffs-Schicht
<code>routes.ts</code>	~102 KB	gesamte API
<code>schema.ts</code>	~73 KB	Datenmodell

Einordnung: Das ist die Größenordnung eines kleinen kommerziellen SaaS, nicht eines Hobby-Projekts.

2. Architektur & Programmaufbau

2.1 Schichten

```
Browser (React SPA, Hash-Routing)
  | fetch + Bearer-Token / Cookie
  ▼
Express API (server/routes.ts, 233 Endpunkte)
  |
  ▼
storage.ts (Daten-/Geschäftslogik-Schicht, ~160 KB)
  | Drizzle ORM
  ▼
SQLite (data.db, WAL-Modus)
```

2.2 Frontend

- **React + TypeScript + Vite**, Routing via **wouter** (`useHashLocation` → läuft auch in iframes / ohne Server-Rewrites).
- **State/Server-Cache**: TanStack React Query (93 Queries, zentrale `queryClient`).
- **UI**: TailwindCSS + **shadcn/ui** (47 Komponenten) — modernes, konsistentes Design-System.
- **i18n**: `i18next` + Browser-Detector, Sprachen **DE/FR/EN/LU**, Persistenz in `localStorage` (`m75_lang`), Fallback DE.
- **Auth-Context**: `lib/auth.tsx` mit Passwort-, Karten- (Random-No) und Admin-Login; Bearer-Token im Speicher (für iframe-Fälle).
- **Fehler-Robustheit**: **ErrorBoundary** (neu, 29.06.) umschließt den Seiteninhalt → kein Totalabsturz mehr, Navigation bleibt nutzbar.

2.3 Backend

- **Node.js + Express + TypeScript**, Einstieg `server/index.ts` .
- **Sicherheits-Middleware** (`security.ts` , ~10 KB): Rate-Limiting (100 Req/15 min), Threat-Detection, Security-Header, Audit-Logging.
- **Spezial-Module**: `flhImport.ts` (handball4all.de), `import.ts` (Excel-Mitgliederimport), `importSboLinks.ts` (SBO-Berichte), `email.ts` (SMTP/Templates), `auth.ts` .
- **ORM**: Drizzle + better-sqlite3; `drizzle-kit push` für Schema-Sync.

2.4 Betrieb / DevOps

- **Docker + docker-compose**, `Dockerfile` , `README-HETZNER.md` → Ziel-Hosting **Hetzner Cloud**.
- **Build**: `tsx script/build.ts` ; Start `node dist/index.cjs` .
- **QA**: statische Screenshots (`qa-*.png`) vorhanden — noch keine automatisierten E2E-Tests.

2.5 Bewertung Aufbau

- **Stark**: klare Schichtung, eine zentrale Storage-Schicht, typisiertes Schema, konsequentes UI-System, durchdachte Auth-Wege, Security-Middleware vorhanden.
- **Schwächen / Schulden**:
- `routes.ts` (102 KB) und `storage.ts` (160 KB) sind **Monolith-Dateien** → schwer wartbar; sollten modular nach Domäne aufgeteilt werden.
- **SQLite** ist für den Pilot ok, aber `data.db-wal` (~4 MB) zeigt aktive Last; für Mehr-Vereins-Betrieb ist die geplante **PostgreSQL-Migration** nötig.
- **Keine automatisierten Tests** (nur Screenshots) → Regressionsrisiko bei 233 Endpunkten.
- **Demo-Passwörter** stehen im Klartext in der Doku → vor Produktiv zwingend rotieren.

3. Visuelles Konzept

- **Design-Sprache:** iOS-/„Apple-style“, aufgeräumt: abgerundete Karten (`rounded-2xl`), Blur-Header (`backdrop-blur-xl`), schwebende Sidebar mit Verlauf (`#002F65` → `#00193a`), Akzentfarbe **Vereinsgelb** `#FFDE00` .
- **Layout:** feste Desktop-Sidebar (268 px) mit Sektionen *Übersicht* / *Verein* / *Sport* / *Verwaltung*; mobil **Sheet-Menü** + **iOS-Tab-Bar** unten; sticky Header mit Seitentitel & Theme-Toggle.
- **Dark/Light Mode** vorhanden (`lib/theme`).
- **Konsistenz:** durch shadcn/ui sehr einheitlich (Buttons, Cards, Inputs, Toasts, Tooltips).
- **Verbesserungspotenzial:**
 - Sidebar wird mit aktuell ~30 Einträgen lang → **Favoriten/„Zuletzt genutzt“** oder kollabierbare Sektionen.
 - Einheitliche **Empty-States & Skeletons** (heute teils nur „Lädt...“-Text).
 - **Globale Suche** (Command-Palette, ⌘K) über Mitglieder/Spiele/Dokumente.
 - Rollen-spezifische **Dashboard-Widgets** (Spieler sieht Spielplan, Kassenwart sieht offene Beiträge).

4. Modul-Status: fertig / ausbaufähig / nur Idee

Legende:  **Fertig & nutzbar** ·  **Funktioniert, ausbaufähig** ·  **Platzhalter/Stub** ·  **Nur Datenmodell (Tabelle ohne UI)** ·  **Nur Plan/Idee**

4.1 Fertig & nutzbar (echter Funktionsumfang, große Seiten)

Modul	Datei	~Zeilen
Login (3 Wege)	Login.tsx	412
Spiele (inkl. FLH-Import)	Matches.tsx	587
Beiträge	Fees.tsx	529
Check-in / Karten-Scan (QR)	CheckIn.tsx	493
Trainingsplan	TrainingSchedules.tsx	413
Statistiken / Berichte	Statistics.tsx	396
Anwesenheit	Attendance.tsx	391
Kalender (ICS)	Calendar.tsx	361
Dokumente	Documents.tsx	328
Mitglieder-Import (Excel)	ImportMembers.tsx	326
E-Mail-Einstellungen	EmailSettings.tsx	312
Spielerstatistiken	PlayerStatistics.tsx	309
Finanzen	Finance.tsx	300

4.2 **Funktioniert, ausbaufähig (solide, aber Luft nach oben)**

Modul	Datei	~Zeilen	Ausbau-Idee
Anmeldungen (intern)	Registrations.tsx	292	Workflow/Statuswechsel, E-Mail-Trigger
Mitglied-Detail	MemberDetail.tsx	289	Lizenz/INAPS/CSMS-Tab, Historie
Nominierungen	Nominations.tsx	280	Drag&Drop-Aufstellung, Verfügbarkeits-Sync
Ankündigungen	Announcements.tsx	250	Zielgruppen, Push/Matrix-Ausspielung
Chat	Chat.tsx	247	echte Matrix-Anbindung statt interner Tabelle
Mitglieder	Members.tsx	228	Gruppen-/Mail-Listen-Generierung
Öffentl. Anmeldung	Registration.tsx	201	Bezahlung, Bestätigungs-Flow
Profil	Profile.tsx	201	2FA, Benachrichtigungs-Einstellungen
Dashboard	Dashboard.tsx	181	rollenbasierte Widgets, KPIs
Website-Hub	Website.tsx	145	Live-Daten-Widgets statt nur iframe
Willkommensmappe	WelcomeMappe.tsx	148	direkte PDF-Erzeugung, Versand an Neu-Mitglied
Teams / Meetings	Teams.tsx / Meetings.tsx	134 / 138	Staffeln, Videokonferenz-Links

4.3 **Platzhalter-Stubs (generisches CRUD-Template, je 89 Zeilen)**

Identische Vorlage (Liste + Anlegen + Löschen, Felder *Name/Beschreibung*) — Funktion „angedeutet“, aber **fachlich noch leer**:

Budget · Duties (Dienste) · Facilities (Hallen) · Gallery · GdprTools · Newsletter · Shop · Sponsors · Waitlist

Diese sind die wahrscheinlichsten Kandidaten für „noch nicht ausgebaut“. Dank ErrorBoundary führen sie nicht mehr zum weißen Bildschirm.

4.4 **Nur Datenmodell vorhanden (Tabelle existiert, UI fehlt)**

Aus `schema.ts` — bereits modelliert, aber **ohne eigene Oberfläche**:

referees , referee_assignments (Schiedsrichter) · inventory_items , inventory_loans (Material) · injuries , rehab_sessions (Verletzung/Reha) · polls , poll_options , poll_votes (Umfragen) · carpools , carpool_passengers (Fahrgemeinschaften) · opponents , opponent_history , match_reports (Scouting) · duty_rosters , duty_swaps (Dienstplan-Tausch) · fan_content , live_ticker · external_calendars , calendar_sync_logs · archive_seasons , archive_teams , archive_members , archive_matches , archive_events , archive_exports (Saison-Archiv) · sepa_mandates , sepa_transactions · family_links , document_expiries , document_signatures , user_notifications .

Empfehlung: Diese Module sind „halbfertig zum Nulltarif“ — UI darauf zu setzen ist schnell und liefert großen Mehrwert.

4.5 **Nur Plan/Idee (kein Code)**

Aus `PROJEKTPLAN.md` : Matrix/Element-Server, n8n-Workflows, Ollama-KI/RAG, Odysseus-Browser-Automation, PPTX-Generator, öffentliche Website-API, PWA/Push, Uptime-Kuma-Monitoring, Mautrix-Bridges (WhatsApp/Signal), Nextcloud, PostgreSQL-Migration.

- **Spielwochen-Recap (News):** wöchentlicher Kurztext über alle Mannschafts-Spiele auf der Website-News-Seite; vom Manager erfasst/verwaltet, optional KI-vorbefüllt. **Geplant ab Saison 2026/2027.**

Erledigt (Juni 2026, Website-Repo): Saison-Spielplan **25/26** als Archiv-Seite `live-center-25-26.html` gesichert (verlinkt aus `statistics-25-26.html`); `live-center.html` auf Struktur für **2026/2027** geleert (FLH-Sync füllt automatisch). Sinnvoll, dies künftig über das App-Archiv-Modul zu automatisieren.

5. Komplexitätsbewertung

Dimension	Bewertung	Begründung
Funktionsumfang	Hoch	35 Seiten, 80 Tabellen, 233 Endpunkte
Code-Qualität (Struktur)	Mittel-Hoch	gute Schichtung, aber Monolith-Dateien
Wartbarkeit	Mittel	<code>routes.ts</code> / <code>storage.ts</code> zu groß; keine Tests
Skalierbarkeit	Mittel	SQLite-Limit; PostgreSQL geplant
Sicherheit	Mittel-Hoch	Middleware vorhanden; Demo-PW & Secrets-Handling offen
Reifegrad UI	Gemischt	Top-Seiten vs. 9 leere Stubs
Doku	Sehr hoch	PROJEKTPLAN + Features-Doku ungewöhnlich gut

6. Meine Anregungen (priorisiert)

6.1 Sofort / geringer Aufwand

1. **Stubs „ehrlich“ kennzeichnen:** Platzhalter-Seiten mit Badge „In Entwicklung“ + kurzer Beschreibung, was kommt (statt scheinbar funktionierendem CRUD).
2. **Globale %K-Suche** über Mitglieder/Spiele/Dokumente.
3. **Rollen-Dashboard:** Widgets je Rolle (offene Beiträge, nächstes Spiel, fällige Lizenzen/INAPS).
4. **Einheitliche Empty-States & Skeleton-Loader.**
5. **Health-Endpoint** `GET /api/health` (DB-Ping, Version, Uptime) als Basis fürs Monitoring.

6.2 Mittlerer Aufwand (großer Hebel, da Tabellen schon da)

1. **Saison-Archiv-UI** auf `archive_*` (einfrieren, abrufen, ZIP/PDF/Excel-Export).
2. **Schiedsrichter-Modul** auf `referees` / `referee_assignments` .
3. **Verletzungs-/Reha-Tracker** (`injuries` / `rehab_sessions`) inkl. CSMS-Schadensfall-Workflow.
4. **Umfragen/Abstimmungen** (`polls`) — z. B. Termin-/Trikot-Voting.
5. **Fahrgemeinschaften** (`carpools`) für Auswärtsspiele.
6. **Benachrichtigungs-Zentrum** auf `user_notifications` (In-App + E-Mail + später Push/Matrix).

6.3 Strategisch

1. **Öffentliche REST-API** (read-only) + Widgets für mersch75.lu (Tabelle, nächste Spiele, Torschützen) — verbindet App & Website automatisch.
2. **PWA** (installierbar, offline Spielplan, Push).

- 3. **Modularisierung** von `routes.ts` / `storage.ts` nach Domänen (Router pro Bereich).
- 4. **Automatisierte Tests** (Vitest + Playwright) für die kritischen Flows.
- 5. **Secrets-Management** (.env + verschlüsselte Felder), Demo-PW raus.

7. KI-, Automatisierungs- & Monitoring-Konzept

Leitlinie aus dem Projekt: **Open Source first, selbst-gehostet, kostenlos, datenschutzkonform (RGPD)**. KI-Daten sollen den Hetzner-Server möglichst **nicht verlassen**.

7.1 Bausteine & ehrliche Einordnung der genannten Tools

Genannt	Was es wirklich ist	Rolle in Vereins-OS
Ollama	Lokale LLM-Laufzeit (llama3, mistral, phi3, qwen, nomic-embed)	Kern der lokalen KI — Chat, RAG, Textgenerierung, Embeddings
Hermes	(a) im Plan: eigener Orchestrierungs-/Monitoring-Layer; (b) Nous-Hermes -Modelle (LLM-Familie, läuft auf Ollama)	(a) Monitoring-Router, (b) konkretes Modell <code>hermes3</code> als Assistent-Modell
Matrix	Offenes, föderiertes, E2E-verschlüsseltes Chat-Protokoll (Server: Synapse)	Selbst-gehostete Kommunikations-Zentrale , Bot-Ausspielung
Element	Offizieller Matrix-Client (Web/Mobile)	Chat-Frontend für Mitglieder
„Elementor“	⚠ Eigentlich ein WordPress-Page-Builder — passt nicht zu einer React-App	Vermutlich Element gemeint; Elementor nur relevant, falls die Website je auf WordPress liefe
„openclaw“	⚠ Kein bekanntes KI-Tool (OpenClaw = Spiel-Engine-Reimplementierung)	Klärungsbedarf . Wahrscheinlich gemeint ist eines von: Open WebUI (Chat-UI für Ollama), Open Interpreter (LLM führt Code/Automation aus), AnythingLLM/LibreChat (RAG-Chat), LocalAI (OpenAI-kompatible lokale API), Flowise (LLM-Flow-BUILDER). Siehe 7.5

7.2 Anwendungsfälle (KI)

- **Regelwerk-/Versicherungs-Assistent (RAG)**: Fragen in DE/FR/LU über FLH-Reglemente, Vereinsstatuten, CSMS/AXA — Antwort **mit Quellenangabe**.
- **Spielbericht-Generator**: aus Statistik automatisch einen Bericht formulieren (Entwurf, Mensch gibt frei).
- **E-Mail-/Newsletter-Entwürfe** auf Knopfdruck, mehrsprachig.
- **Natürlichsprachige Datenabfrage**: „Wie viele Tore hat Spieler X diese Saison?“ → KI übersetzt in DB-Query (read-only, abgesichert).
- **Finanz-Anomalie-Erkennung**: ungewöhnliche Buchungen/offene Beiträge melden (Monitoring-KI).
- **Onboarding-Bot**: beantwortet Fragen neuer Mitglieder auf Basis der Willkommensmappe.

7.3 Automatisierung (n8n + Cron + Browser-Automation)

- **n8n** (self-hosted) als visueller Workflow-Motor: FLH-Ergebnis → DB → Matrix/E-Mail; Neumitglied → Willkommensmail → Beitragsrechnung; Saisonende → Archiv → Bericht.
- **Playwright** („Odysseus“): SBO-Berichte/Live-Daten automatisch abholen, Tabellen-Screenshots für Social Media.
- **Cron-Jobs**: Backups, Geburtstags-/INAPS-Erinnerungen, Wochenbericht.

7.4 Monitoring (der „Hermes-Layer“)

- **Uptime Kuma** (Docker): Erreichbarkeit, Zertifikate, Heartbeats.
- **App-seitig:** `GET /api/health` + Audit-Log-Auswertung; Alarme via E-Mail **und** Matrix-Raum `#monitoring`.
- **Optionale KI-Schicht:** tägliche KI-Zusammenfassung der Logs („Was war heute auffällig?“).

7.5 Vorgeschlagene KI-Architektur mit Update-Möglichkeit (wichtig!)

Damit Modelle/Anbieter **ohne Code-Änderung** wechsel- und aktualisierbar sind, ein **Adapter-Muster + Admin-UI**:

Datenmodell (neu):

```
ai_providers
├── id
├── type          (ollama | openai | anthropic | localai | custom)
├── label
├── base_url      (z.B. http://localhost:11434)
├── api_key_enc   (verschlüsselt; leer bei lokal)
├── model         (z.B. llama3.1, hermes3, mistral)
├── embed_model   (z.B. nomic-embed-text)
├── enabled       (bool)
├── is_default    (bool)

ai_jobs          (Audit: Prompt, Modell, Tokens, Dauer, Nutzer, Kosten)
ai_documents     (RAG-Quellen: Datei, Chunks, Embeddings)
```

Server-Abstraktion: ein `aiService` mit **OpenAI-kompatibler Schnittstelle** — Ollama, LocalAI, OpenAI und Anthropic sprechen (fast) dasselbe Format, dadurch ist der Anbieter austauschbar.

Admin-Seite „KI-Einstellungen“:

- Anbieter/Modell wählen & testen, Standard setzen.
- **Modelle live verwalten:** `GET /api/ai/models` (listet installierte Ollama-Modelle), `POST /api/ai/pull` (lädt/aktualisiert ein Modell), Fortschrittsanzeige.
- API-Keys **verschlüsselt** speichern (nie im Klartext/Frontend).
- **Auto-Update:** Cron/n8n zieht regelmäßig `ollama pull <model>` für gepinnte Modelle; Versions-Pinning verhindert Überraschungen.

RAG-Pipeline:

```
PDFs/Statuten → Chunking → Embeddings (nomic-embed-text @ Ollama)
  → Vektor-Store (sqlite-vec ODER Chroma)
  → Retrieval → Prompt + Quellen → LLM (Ollama/Hermes) → Antwort mit Zitaten
```

Sicherheit: KI nur lesend auf Vereinsdaten; jede KI-Aktion in `ai_jobs` protokolliert; PII-Minimierung in Prompts; lokale Modelle bevorzugt (kein Datenabfluss).

7.6 Empfohlene erste KI-Schritte (Reihenfolge)

1. Ollama auf Hetzner + `GET /api/health` + `ai_providers`-Tabelle.
 2. Admin-„KI-Einstellungen“ (Anbieter/Modell wählen, `pull`, testen).
 3. RAG-Assistent über FLH-PDFs (größter, sichtbarster Nutzen).
 4. Spielbericht- & E-Mail-Generator.
 5. n8n + erste Automationen; Uptime Kuma + Matrix-Alarme.
-

8. Risiken & nächste sinnvolle Schritte

Risiken: Monolith-Dateien (Wartung), SQLite-Grenzen, fehlende Tests, Demo-Secrets, Reife-Gefälle der UI.

Empfohlene nächste 5 Schritte:

1. Stubs kennzeichnen + 2–3 davon auf vorhandene Tabellen real ausbauen (z. B. Sponsoren, Galerie, Saison-Archiv).
 2. GET /api/health + Uptime-Kuma-Monitoring.
 3. Öffentliche Read-API + 1 Website-Widget (Ligatabelle) als sichtbarer Quick-Win.
 4. KI-Fundament (Ollama + ai_providers + Admin-UI mit Update/Pull).
 5. Secrets bereinigen, erste Vitest/Playwright-Tests für kritische Flows.
-

Hinweis: Dieser Bericht basiert auf statischer Code-Analyse vom 29.06.2026. Die mit ⚠ markierten Tool-Namen („Elementor“, „openclaw“) sollten kurz bestätigt werden, damit die KI-Integration den richtigen Baustein nutzt.